

# Gaussian Blur

### Limits

Time 1500 ms

Memory 1024 MiB

Source size 64 KiB

Points 100

### Submit

Code

C++ ▾

Submit

| Verdict      | Time    | Memory | Language | Points | Samples |
|--------------|---------|--------|----------|--------|---------|
| wrong_answer | 1983 ms | 5 MiB  | cpp      | 0/100  | ✓       |
| wrong_answer | 1980 ms | 5 MiB  | cpp      | 0/100  | ✓       |
| wrong_answer | 1980 ms | 5 MiB  | cpp      | 10/100 | ✓       |
| ▼ Expand     |         |        |          |        |         |

## Description

You are given an  $M \times N$  matrix with  $M$  rows and  $N$  columns whose entries are integers. The rows and columns of the matrix are indexed from 0. That means the top row has index 0, the bottom row has index  $M - 1$ . The leftmost column has index 0 and the rightmost column has index  $N - 1$ .

Denote by  $(i, j)$  the cell of the matrix which is located in the  $i$ -th row and  $j$ -th column. A square whose top left cell is  $(i - R, j - R)$  and its bottom right cell is  $(i + R, j + R)$  is called a **square with center  $(i, j)$  and radius  $R$** .

Below you can see a matrix with  $M = 9, N = 10$ . In green we have a **square with center (2, 2) and radius 1**. In blue we have a **square with center (5, 6) and radius 2**.

|        |  |  |  |  |  |  |  |  |        |
|--------|--|--|--|--|--|--|--|--|--------|
| (0,0)  |  |  |  |  |  |  |  |  | (0, 9) |
|        |  |  |  |  |  |  |  |  |        |
|        |  |  |  |  |  |  |  |  |        |
|        |  |  |  |  |  |  |  |  |        |
|        |  |  |  |  |  |  |  |  |        |
|        |  |  |  |  |  |  |  |  |        |
|        |  |  |  |  |  |  |  |  |        |
|        |  |  |  |  |  |  |  |  |        |
|        |  |  |  |  |  |  |  |  |        |
| (8, 0) |  |  |  |  |  |  |  |  | (8, 9) |

Call the given matrix  $A$ . You are also given a positive integer  $C$ . For a square with center  $(i_0, j_0)$  and radius  $R$ , we define its **Gaussian blur** as the sum of all values  $A[i][j] \cdot C^{|i-i_0|+|j-j_0|}$ , where  $(i, j)$  go through all the cells of the mentioned square.

Below is a very rough piece of code that calculates that sum:

```
for(int i = -R; i <= R; ++i){
    for(int j = -R; j <= R; ++j){
        sum = sum + A[i_0 + i][j_0 + j] * pow(C, abs(i) + abs(j));
    }
}
```

Your task is to calculate the Gaussian blur of several such squares, all having the same radius  $R$ . As the resulting number can be very large, print its value modulo  $1000000007 = 10^9 + 7$ , which is a prime number. Be careful to use **long long** to avoid overflow where needed.

It is guaranteed that we will never consider a square which is not completely inside our matrix.

## Input Data

In the first line, there are four integers  $M, N, R$ , and  $C$ , in that order.

Numbers  $M$  and  $N$  represent the number of rows and columns of our matrix and  $3 \leq M, N \leq 1000$ . Number  $1 \leq R \leq 200$  is the radius of the squares which will be considered (same  $R$  for all squares). Number  $1 \leq C \leq 1000$  is the number  $C$  from the problem statement (same  $C$  for all squares).

In each of the next  $M$  lines there are  $N$  integers  $0 \leq x \leq 1000$ , the values in the cells of our matrix.

In the next line, there is a single integer  $1 \leq T \leq 100000$ , the number of test cases.

In each of the next  $T$  lines, there are two integers  $R \leq i \leq M - R - 1$  and  $R \leq j \leq N - R - 1$ , representing the center of a square with center  $(i, j)$  and radius  $R$ .

## Output Data

Print  $T$  lines, one for each input test case. In each line, print one non-negative integer, the Gaussian blur of the corresponding square with radius  $R$ , modulo  $1000000007 = 10^9 + 7$ .

## Scoring

First test data cluster is worth 10 points and  $M, N \leq 10, R \leq 3, C \leq 2, T \leq 10$ .

Second test data cluster is worth 5 points and  $T \leq 100$ .

Third test data cluster is worth 5 points and  $M, N \leq 100, R \leq 40$ .

Fourth test data cluster is worth 20 points and  $C = 1$ .

Fifth test data cluster is worth 30 points and  $M, N \leq 500, R \leq 100$ .

Sixth test data cluster is worth 30 points and there are no additional constraints.

## Example

### Input:

```
10 10 2 2
7 5 6 3 7 9 3 2 8 2
1 7 1 1 5 9 1 8 7 6
5 0 3 2 2 4 1 7 8 0
6 5 9 3 8 4 4 5 3 9
1 4 6 9 0 1 5 2 2 0
8 6 2 1 8 7 6 1 8 3
6 7 4 4 1 9 1 9 7 2
2 4 0 8 0 3 6 0 3 6
0 7 7 8 4 0 2 3 6 1
0 7 9 6 3 7 2 3 5 6
2
```

```
2 4
```

```
3 2
```

## Output:

```
768
```

```
707
```

## Explanation:

The first square in consideration is:

```
6 3 7 9 3
```

```
1 1 5 9 1
```

```
3 2 2 4 1
```

```
9 3 8 4 4
```

```
6 9 0 1 5
```

$$\begin{aligned}
 \text{sum} &= 6 * 2^4 + 3 * 2^3 + 7 * 2^2 + 9 * 2^1 + 3 * 2^0 + \\
 &+ 1 * 2^4 + 1 * 2^3 + 5 * 2^2 + 9 * 2^1 + 1 * 2^0 + \\
 &+ 3 * 2^4 + 2 * 2^3 + 2 * 2^2 + 4 * 2^1 + 1 * 2^0 + \\
 &+ 9 * 2^4 + 3 * 2^3 + 8 * 2^2 + 4 * 2^1 + 4 * 2^0 + \\
 &+ 6 * 2^4 + 9 * 2^3 + 0 * 2^2 + 1 * 2^1 + 5 * 2^0 = \\
 &= 768
 \end{aligned}$$

The second square in consideration is:

```
1 7 1 1 5
```

```
5 0 3 2 2
```

```
6 5 9 3 8
```

```
1 4 6 9 0
```

```
8 6 2 1 8
```

$$\begin{aligned}
 \text{sum} &= 1 * 2^4 + 7 * 2^3 + 1 * 2^2 + 1 * 2^1 + 5 * 2^0 + \\
 &+ 5 * 2^4 + 0 * 2^3 + 3 * 2^2 + 2 * 2^1 + 2 * 2^0 + \\
 &+ 6 * 2^4 + 5 * 2^3 + 9 * 2^2 + 3 * 2^1 + 8 * 2^0 + \\
 &+ 1 * 2^4 + 4 * 2^3 + 6 * 2^2 + 9 * 2^1 + 0 * 2^0 + \\
 &+ 8 * 2^4 + 6 * 2^3 + 2 * 2^2 + 1 * 2^1 + 8 * 2^0 = \\
 &= 707
 \end{aligned}$$